

Algoritmi e Strutture Dati – 26/8/2024 – Parte A

Esercizio -1 Iscriverti allo scritto entro la scadenza.
In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

A1 – Complessità – Punti ≥ 8

Supponiamo di dover scegliere tra tre algoritmi per risolvere un problema di dimensione n :

- l'algoritmo A divide ricorsivamente il problema in 5 sottoproblemi di dimensione $n/2$, per poi combinare le soluzioni parziali in tempo lineare;
- l'algoritmo B divide ricorsivamente il problema in 2 sottoproblemi di dimensione $n - 2$, per poi combinare le soluzioni parziali in tempo costante;
- l'algoritmo C divide ricorsivamente il problema in 9 sottoproblemi di dimensione $n/3$, per poi combinare le soluzioni parziali in tempo $O(n^2)$.

Quale algoritmo è preferibile utilizzare e perché?

A2 – Alberi isobati – Punti ≥ 10

Un albero si dice *isobato* se tutte le sue foglie hanno la stessa profondità.¹

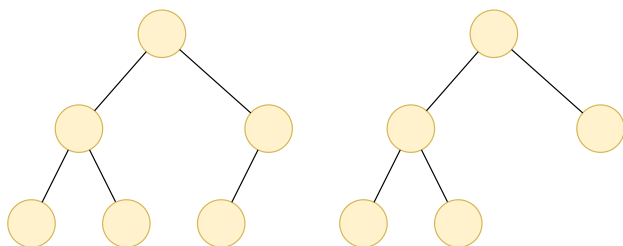
Scrivere un algoritmo

boolean isIsobath(TREE T)

che prende in input un albero binario T e restituisce **true** se T è un albero isobato, **false** altrimenti.

Discutere informalmente la correttezza dell'algoritmo proposto e la sua complessità computazionale.

Nella figura seguente, l'albero a sinistra è isobato, quello a destra no.



A3 – Where's negative? – Punti ≥ 12

Sia $f : \mathbb{N}^+ \rightarrow \mathbb{Z}$ una funzione *monotona decrescente*, che restituisce un valore intero $f(k)$ per ogni naturale $k > 0$ e tale per cui $f(1) > 0$. Si assuma che il costo di esecuzione di f sia $O(1)$.

Scrivere un algoritmo

int findNegative()

che restituisca il più piccolo valore n tale per cui $f(n) < 0$. Essendo monotona decrescente, tale valore esiste sicuramente.

Discutere informalmente la correttezza dell'algoritmo proposto e la sua complessità computazionale, calcolata in funzione del valore n per cui $f(n) < 0$. Algoritmi di costo lineare verranno valutati 30%, algoritmi di costo superiore non verranno considerati.

¹<https://it.wikipedia.org/wiki/Isobata>

Algoritmi e Strutture Dati – 26/8/2024 – Parte B

Esercizio -1 Iscriverti allo scritto entro la scadenza. In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

B1 – Connect-x – Punti ≥ 8

Si consideri una matrice booleana A di dimensione $n \times n$. Scrivere un algoritmo

```
int longestLine(boolean[][] A, int n)
```

che restituisca la lunghezza della più lunga sequenza contigua (orizzontale o verticale) di valori **true**.

Discutere informalmente la correttezza dell'algoritmo proposto e la sua complessità computazionale.

Nella figura seguente, la linea evidenziata in giallo è la più lunga sequenza contigua di valori **true**, quindi l'algoritmo dovrà restituire 5. Si noti che le prime due righe contengono più di 5 valori **true**, che però non sono contigui.

T	T	F	T	T	T	T
T	T	F	T	T	T	T
F	F	T	F	F	F	F
T	T	T	T	F	T	F
T	F	T	F	F	F	F
F	F	T	F	T	T	T
F	T	T	T	F	F	F

B2 – Formazione stringhe – Punti ≥ 10

Consideriamo un processo di formazione di stringhe in cui è possibile fare le seguenti mosse:

- aggiungere un carattere in fondo alla stringa
- duplicare la stringa esistente

Scrivere un algoritmo

```
int minMoves(item[] S, int n)
```

che prenda in input una stringa S lunga n caratteri e restituisca il numero minimo di mosse necessarie per formare S seguendo le regole appena descritte.

Discutere informalmente la correttezza dell'algoritmo proposto e la sua complessità computazionale.

Per esempio, la stringa "tatami" può essere ottenuta in due modi:

• $\emptyset \rightarrow "t" \rightarrow "ta" \rightarrow "tat" \rightarrow "tata" \rightarrow "tatam" \rightarrow "tatami"$, per un totale di 6 mosse

• $\emptyset \rightarrow "t" \rightarrow "ta" \rightarrow "tata" \rightarrow "tatam" \rightarrow "tatami"$, per un totale di 5 mosse

In questo caso, l'algoritmo dovrà restituire 5.

La stringa "babababa", invece, può essere ottenuta nel modo seguente, che rappresenta il numero minimo di mosse:

• $\emptyset \rightarrow "b" \rightarrow "ba" \rightarrow "baba" \rightarrow "babababa"$

In questo caso, l'algoritmo dovrà restituire 4.

B3 – Zigzagghiamo! – Punti ≥ 12

Una sequenza di interi $X = x_1, x_2, \dots, x_n$ si dice a zig-zag se

- $x_i < x_{i+1}$ se i è dispari
- $x_i > x_{i+1}$ se i è pari

Scrivere un algoritmo

```
printZigZag(int[] A, int n)
```

che prenda in input un vettore A di dimensione n e stampi tutte le sottosequenze a zig-zag contenute nel vettore A . Si noti che il controllo di parità degli indici va fatto sugli indici della sottosequenza, non sugli indici della stringa originale.

Discutere informalmente la correttezza dell'algoritmo proposto e la sua complessità computazionale.

Per esempio, $A = [1, 3, 6, 4, 5]$ dà origine a questo output, non necessariamente in quest'ordine:

$[], [5], [4], [4, 5], [6], [3], [3, 5], [3, 4], [3, 6], [3, 6, 5], [3, 6, 4], [3, 6, 4, 5], [1], [1, 5], [1, 4], [1, 6], [1, 6, 5], [1, 6, 4], [1, 6, 4, 5], [1, 3]$